

Metodi avanzati di programmazione

Alessandro Carella

Indice

1	Astrazione nella progettazione	2
1.1	Astrazione funzionale	3
1.1.1	Specifiche semantiche	3
1.1.2	Limiti dell'astrazione funzionale	4
1.2	Astrazione dati	4
1.2.1	Information Hiding	4
1.2.2	Incapsulamento	5
1.3	Specifica sintattica e semantica	6
1.3.1	Specifiche assiomatiche	7
1.3.2	Specifiche algebriche	7
2	L'astrazione nella programmazione	13
2.1	Astrazione di funzione	14
2.1.1	Anomalie di progetto dei linguaggi	14
2.2	Astrazione di procedura	16
2.3	Astrazione di controllo	17
2.4	Astrazione di selettore	20
2.5	Astrazione dati	22
2.5.1	Tecniche di programmazione a supporto dell'astrazione dati	22
2.6	Astrazione nella dichiarazione di modulo	32
3	Paradigma Object Oriented	34
3.1	Gli oggetti	34
3.1.1	Identificatore di oggetto	35
3.2	Le classi	35

Capitolo 1

Astrazione nella progettazione

In ambito scientifico la parola "astrarre" significa cambiare la rappresentazione di un problema e questo serve a concentrarsi su aspetti *rilevanti* dimenticando gli elementi *incidentali*.

Il termine astrazione sotto-intende:

- un **processo**: l'estrazione delle informazioni essenziali e rilevanti per un particolare scopo, ignorando il resto dell'informazione;
- un'**entità**: una descrizione semplificata di un sistema che enfatizza alcuni dei dettagli o proprietà trascurandone altri.

In programmazione, l'astrazione allude alla distinzione che si fa tra:

- *cosa* (*what*) un programma fa;
- *come* (*how*) viene implementato.

Per l'utente del codice l'essenziale è cosa fa il codice mentre non è interessato ai dettagli implementativi.

Nell'informatica l'astrazione è un concetto importante, in quanto, i sistemi software diventano sempre più complessi e per *padroneggiare la complessità* è necessario concentrarsi solo sui pochi aspetti che più interessano in un certo contesto e ignorarne altri.

L'astrazione permette ai progettisti di sistemi software di risolvere problemi complessi in modo organizzato e gestibile.

Esistono diverse tipologie di astrazione: l'*astrazione funzionale* e l'*astrazione dati*.

1.1 Astrazione funzionale

L'astrazione funzionale si riferisce alla *progettazione del software*, e in particolare alla possibilità di specificare un modulo software che **trasforma** dei dati di input in dati di output nascondendo i dettagli algoritmici della trasformazione. Sinteticamente, un modulo software deve trasformare un input in un output, cioè deve calcolare una funzione. I dettagli della trasformazione non sono visibili al consumatore del modulo, esso conosce solo le corrette convenzioni di chiamata, la *specificata sintattica*, e cosa fa il modulo, la *specificata semantica* e il consumatore deve fidarsi del risultato.

Un esempio di modulo software è il modulo che realizza un operatore per il calcolo del fattoriale. La *specificata sintattica* indica il nome del modulo, il tipo di dato in input e il tipo di risultato, in modo da permettere la corretta chiamata del modulo. Per la funzione del fattoriale:

$$fatt(intero) \rightarrow intero$$

La *specificata semantica* indica la trasformazione operata, cioè la funzione calcolata:

$$fatt(n) = \begin{cases} n(n-1) \cdots 2 \cdot 1 & n \geq 1 \\ 1 & n = 0 \end{cases}$$

Come la trasformazione è calcolata o realizzata non è noto al fruitore del modulo.

1.1.1 Specifiche semantiche

Un modo per specificare la semantica di un modulo è quello di esprimere, mediante due *predicati*, la relazione che lega i dati di ingresso e ai dati di uscita: se il primo predicato, detto *precondizione*, è vero sui dati di ingresso e se il programma termina su quei dati, allora il secondo, detto *postcondizione*, è vero sui dati di uscita.

Queste specifiche semantiche sono dette **assiomatiche**.

Nell'esempio del fattoriale:

- **Precondizione:** $n \in \mathbb{N}$
- **Postcondizione:** $fatt(n) = \begin{cases} n(n-1) \cdots 2 \cdot 1 & n \geq 1 \\ 1 & n = 0 \end{cases}$

Esercizio 1.1.1

Un modulo ha in input un intero x e un vettore $\mathbf{A}$ di n interi e restituisce un intero p e un vettore $\mathbf{B}$ di n interi.

Siano date le seguenti:

- **Precondizione:** $\{n > 0 \wedge \forall i \in [1, n] A[i] \in Z\}$
- **Postcondizione:** $\{\forall i \in [1, n] \exists j \in [1, n] B[i] = A[j] \wedge \forall i \in [1, p - 1] B[i] \leq x \wedge \forall i \in [p + 1, n] B[i] > x\}$

Questa è la specifica semantica del modulo dell'*x partition del quick sort*

1.1.2 Limiti dell'astrazione funzionale

L'astrazione funzionale non permette di progettare, e quindi sviluppare, moduli software *invarianti ai cambiamenti nei dati*, sono invariant solo ai cambiamenti nei processi di trasformazione che operano.

Questo rende difficoltosa la manutenzione delle soluzioni progettate ed è inappropriata per lo sviluppo di soluzioni a problemi complessi.

1.2 Astrazione dati

Alla base dell'*astrazione dati* c'è il principio che *non si può accedere direttamente alla rappresentazione di un dato, qualunque esso sia, ma solo attraverso un insieme di operazioni considerate lecite* e questo è detto **principio dell'astrazione dati**.

Un vantaggio dell'astrazione dati è che un cambiamento nella rappresentazione del dato si ripercuoterà solo sulle operazioni lecite, che potrebbero subire delle modifiche, mentre non inficerà il codice che utilizza il dato astratto.

1.2.1 Information Hiding

In generale un principio di astrazione suggerisce di *occultare l'informazione*, l'**information hiding**, sulla rappresentazione del dato sia perchè non necessaria al fruitore dell'entità astratta e sia perchè la sua rivelazione creerebbe delle inutili dipendenze che comprometterebbero l'invarianza ai cambiamenti.

Il principio dell'astrazione funzionale suggerisce di occultare i dettagli del processo di trasformazione. Il principio dell'astrazione dati identifica nella rappresentazione del dato l'informazione da nascondere.

1.2.2 Incapsulamento

L'**incapsulamento** è una tecnica di progettazione consistente nell'impacchettare una collezione di entità, creandone una barriera concettuale. Come l'astrazione, l'incapsulamento sottintende:

- un **processo**: l'impacchettamento;
- un' **entità**: il pacchetto ottenuto.

Ad essa corrispondono tecniche di programmazione che consentono l'incapsulamento dei dati.

L'incapsulamento non dice come devono essere le “pareti” del pacchetto, che potranno essere:

- **trasparenti**: permettendo di vedere tutto ciò che è stato impacchettato;
- **traslucide**: permettendo di vedere in modo parziale il contenuto;
- **opache**: nascondendo tutto il contenuto del pacchetto.

La combinazione del principio dell'astrazione dati con la tecnica dell'incapsulamento suggerisce che:

1. La rappresentazione del dato va nascosta;
2. L'accesso al dato deve passare solo attraverso operazioni lecite;
3. Le operazioni lecite, che ovviamente devono avere accesso all'informazione sulla rappresentazione del dato, vanno impacchettate con la rappresentazione del dato stesso.

L'astrazione dati ricalca ed estende quella funzionale. Attualmente la possibilità effettuare astrazione di dati è considerata importante almeno quanto quella di definire nuovi operatori con astrazioni funzionali. L'esperienza ha infatti dimostrato che la scelta delle strutture di dati è il primo passo sostanziale per un buon risultato dell'attività di programmazione.

L'astrazione funzionale stimola gli sforzi per evidenziare operazioni ricorrenti o comunque ben caratterizzate all'interno della soluzione di un dato problema.

L'astrazione di dati stimola in più gli sforzi per individuare l'organizzazione dei dati più consone alla soluzione del problema. Si è passati quindi da una progettazione *function centered* ad una *data centered*.

In generale, le astrazioni supportano la separazione dei diversi interessi di:

- **Utenti:** interessati a cosa si astrae (*what*);
- **Implementatori:** interessati a come (*how*) si realizza.

Per questa ragione una definizione di astrazione ha sempre due componenti:

- la **specificità**;
- la **realizzazione**.

Per descrivere una specificità occorre ricorrere a dei *linguaggi di specificità*, che sono diversi dai linguaggi usati per descrivere le realizzazioni delle astrazioni.

1.3 Specificità sintattica e semantica

La specificità potrà poi essere:

- **Sintattica:** stabilisce quali identificatori sono associati all'astrazione;
- **Semantica:** definisce il risultato della computazione inclusa nell'astrazione.

L'efficacia di un'astrazione può essere migliorata mediante l'uso di **parametri** per la comunicazione con l'ambiente esterno. Quando un'astrazione è chiamata, ciascun parametro formale verrà associato in qualche modo al corrispondente argomento.

Anche un'astrazione dati è costituita da una **specificità** e da una **realizzazione**:

- la **specificità** consente di descrivere un nuovo dato e gli operatori che ad esso sono applicabili;
- la **realizzazione** stabilisce come il nuovo dato e i nuovi operatori vengono ricondotti ai dati e agli operatori già disponibili.

Chi intende usare i nuovi dati con i nuovi operatori nella scrittura dei suoi programmi sarà tenuto a conoscere la specificità dell'astrazione dei dati, ma potrà astrarre delle tecniche utilizzate per la realizzazione.

I linguaggi di specificità per astrazione dati più noti sono due:

1. il linguaggio logico-matematico usato nelle asserzioni, chiamate **specifiche assiomatiche**;
2. il linguaggio dell'algebra usato nelle equazioni definite fra gli operatori specificati nel dato astratto, chiamate **specifiche algebriche**.

1.3.1 Specifiche assiomatiche

Un linguaggio formale per la specifica di un dato astratto è costituito dalla notazione logico-matematica delle **asserzioni**. In questo caso si parla di specifica assiomatica.

Una specifica assiomatica è composta da:

- una specifica **sintattica**, detta *signature*, che fornisce:
 - l'elenco dei nomi dei domini e delle operazioni specifiche del tipo;
 - i domini di partenza e di arrivo per ogni nome di operatore.
- una specifica **semantica** che associa:
 - un insieme ad ogni tipo introdotto nella specifica sintattica;
 - una funzione ad ogni nome di operatore, esplicitando le condizioni sui domini di partenza e di arrivo:
 - * *precondizione*, che definisce quando l'operatore è applicabile;
 - * *postcondizione*, che stabilisce la relazione tra argomenti e risultato.

Limiti delle specifiche assiomatiche Si noti che il metodo di specifica assiomatica è preciso nella definizione della specifica sintattica, mentre è piuttosto informale per altri aspetti, infatti a volte si ricorre al linguaggio naturale per semplicità. Pertanto esso non consente di caratterizzare precisamente un dato astratto. In particolare, non consente di definire i valori che possono essere generati mediante l'applicazione di operatori, e non consente di stabilire quando l'applicazione di diverse sequenze di operatori porta al medesimo valore. Questo problema è superato dalle specifiche algebriche.

1.3.2 Specifica algebriche

Le specifiche algebriche si basano sull'algebra, piuttosto che sulla logica. Essenzialmente definiscono un dato astratto come un'**algebra eterogenea**, ovvero come una collezione di diversi insiemi su cui sono definite diverse operazioni. Le algebre tradizionali sono omogenee. Un'algebra omogenea consiste in un unico insieme e diverse operazioni.

Ad esempio, l'insieme $\mathbb{Z}$ con le operazioni di addizione e moltiplicazione è un

algebra omogenea.

Invece, se consideriamo un alfabeto Σ e consideriamo Σ^* l'insieme di tutte le stringhe, inclusa quella vuota, costruite con i simboli di Σ .

Σ^* con le operazioni di concatenazione e calcolo della lunghezza non sono un'algebra omogenea, visto che il codominio dell'operazione di calcolo della lunghezza è $\mathbb{N}$ e non Σ^* . Quindi, quest'algebra consiste di due insiemi, stringhe e interi, su cui sono definite le operazioni di concatenamento e lunghezza delle stringhe.

Una specifica algebrica consiste di tre parti:

1. **Sintattica:** elenca i nomi del tipo, le sue operazioni e il tipo degli argomenti delle operazioni. Se un'operazione è una funzione allora è specificato il codominio della funzione;
2. **Semantica:** consiste di un insieme di equazioni algebriche che descrivono in modo indipendente dalla rappresentazione le proprietà delle operazioni;
3. **Di restrizione:** stabilisce varie condizioni che devono essere soddisfatte o prima che siano applicate le operazioni o dopo che esse siano state completate.

A volte si possono trovare le specifiche di restrizione in quelle semantiche.

Uno degli aspetti più interessanti delle specifiche algebriche è la semplicità del linguaggio di specifica rispetto ai linguaggi di programmazione procedurale. Infatti, il linguaggio di specifica consiste di solo cinque primitive:

1. composizione funzionale;
2. relazione di equaglianza;
3. la costante *true*;
4. la costante *false*;
5. un numero illimitato di variabili libere.

La funzione *if then else* può essere facilmente descritta dalle seguenti equazioni:

$$if\ then\ else\ (true, q, r) = q$$

$$if\ then\ else\ (false, q, r) = r$$

Questa funzione è così importante che si assume già data come operatore infisso:

$$if\ then\ q\ else\ r$$

Inoltre si assume che sono predefiniti i valori interi e booleani

Esempio : Specifica algebrica di Pila

- **Specifica sintattica**

sorts: stack, item, boolean

operations:

- $\text{newstack}() \rightarrow \text{stack}$
- $\text{push}(\text{stack}, \text{item}) \rightarrow \text{stack}$
- $\text{pop}(\text{stack}) \rightarrow \text{stack}$
- $\text{top}(\text{stack}) \rightarrow \text{item}$
- $\text{isnew}(\text{stack}) \rightarrow \text{boolean}$

Ogni insieme è detto un *sort* dell'algebra eterogenea. I *sort* item e boolean sono *ausiliari* alla definizione di stack.

- **Specifica semantica**

declare *stk*: stack, *i*: item

- $\text{pop}(\text{push}(\text{stk}, i)) = \text{stk}$
- $\text{top}(\text{push}(\text{stk}, i)) = i$
- $\text{isnew}(\text{newstack}) = \text{true}$
- $\text{isnew}(\text{push}(\text{stk}, i)) = \text{false}$

- **Specifica di restrizione**

- $\text{pop}(\text{newstack}) = \text{error}$
- $\text{top}(\text{newstack}) = \text{error}$

dove *error* è un elemento speciale indefinito.

Ovviamente avremmo potuto scrivere molte altre equazioni, come:

$$\text{isnew}(\text{pop}(\text{push}(\text{newstack}, i))) = \text{true}$$

che evidenzia come il predicato *isnew* sia vero anche per quelli stack ottenuti inserendo un generico elemento *i* in un **nuovo** stack e poi rimuovendolo.

Tuttavia questa equazione è ridondante, poichè è ricavabile da altre scritte in precedenza. Infatti, grazie alla prima e alla quarta equazione possiamo scrivere $\text{isnewstack}(\text{pop}(\text{push}(\text{newstack}, i))) = \text{isnew}(\text{newstack}) = \text{true}$.

Nelle specifiche semantiche è importante indicare l'insieme minimale di equazioni, dette *assiomi*, a partire dalle quali possiamo derivare tutte le altre. Le specifiche semantiche si diranno **incomplete** se non permetteranno di derivare tutte le verità desiderate dell'algebra specificate.

Le specifiche semantiche si diranno **inconsistenti** se permetteranno di derivare delle equazioni indesiderate, cioè considerate false. Le specifiche semantiche si diranno **ridondanti** se alcune delle equazioni sono ricavabili dalle altre.

Scrivere delle specifiche semantiche complete, consistenti e non ridondanti può non essere un compito semplice. Per questo conviene introdurre una **metodologia**, che si basa sulla distinzione degli operatori di un dato astratto in:

- **costruttori**, che creano o istanziano il dato astratto;
- **osservazioni**, che ritrovano informazioni sul dato astratto.

Il comportamento di un'astrazione dati può essere specificata riportando il valore di ciascuna osservazione applicata a ciascun costruttore. Questa informazione è organizzata in modo naturale in una matrice, con i costruttori lungo una dimensione e le osservazioni lungo l'altra.

Un esempio di costruttori e osservazioni per la pila è il seguente.

<i>osservazioni</i>	<i>Costruttore di stk'</i>	
	<i>newstack</i>	<i>push(stk, i)</i>
<i>pop(stk')</i>	error	stk
<i>top(stk')</i>	error	i
<i>isnew(stk')</i>	true	false

Tutte le osservazioni viste finora sono unarie, nel senso che esse osservano un singolo valore del dato astratto, ma spesso, è necessario disporre di osservazioni più complesse. Per confrontare due valori è necessario osservare due istanze del dato astratto e questo complica la specifica, perchè il valore dell'osservazione dev'essere definito per tutte le **combinazioni di costruttori** possibili per i valori astratti che si devono confrontare.

Esempio : Predicato *equal*

Consideriamo il predicato *equal(l, m)* che è vero solo se le due pile contengono gli stessi elementi nello stesso ordine.

<i>Costruttore di m</i>	<i>Costruttore di l</i>	
	<i>newstack</i>	<i>push(stk, i)</i>
<i>newstack</i>	true	false
<i>push(stk', i')</i>	false	i=i' and equal(stk,stk')

Questa tabella può essere vista come l'aggiunta di una terza dimensione alla tabella di base per le osservazioni unarie.

Questa terza dimensione può essere rimossa andando a classificare esplicitamente solo il primo argomento dell'osservazione e referenziando in modo astratto il secondo argomento mediante una *variabile libera*.

<i>osservazione</i>	<i>Costruttore di stk'</i>	
	<i>newstack</i>	<i>push(stk, i)</i>
<i>equal(stk', m)</i>	isnew(m)	not isnew(m) and i=top(m) and equal(stk,pop(m))

Esempio : Specifica algebrica della coda

- **Specifica sintattica**

sorts: queue, item, boolean

operations:

- newq() → queue
- addq(queue, item) → queue
- deleteq(queue) → queue
- frontq(queue) → item
- isnewq(queue) → boolean

- **Specifica semantica**

declare q : *queue*, i : *item*

- $\text{deleteq}(\text{addq}(q, i)) = \text{if } \text{isnewq}(q) \text{ then } \text{newq} \text{ else } \text{addq}(\text{deleteq}(q), i)$
- $\text{frontq}(\text{addq}(q, i)) = \text{if } \text{isnewq}(q) \text{ then } i \text{ else } \text{frontq}(q)$
- $\text{isnew}(\text{newq}) = \text{true}$
- $\text{isnew}(\text{addq}(q, i)) = \text{false}$

- **Specifica di restrizione**

- $\text{deleteq}(\text{newq}) = \text{error}$
- $\text{frontq}(\text{newq}) = \text{error}$

Se volessi rappresentare costruttori e osservazioni mediante una tabella, il risultato è il seguente.

osservazioni	<i>Costruttore di q'</i>	
	<i>newq</i>	<i>addq(q, i)</i>
<i>isnewq(q')</i>	true	false
<i>frontq(q')</i>	error	if <i>isnewq(q)</i> then <i>i</i> else <i>frontq(q)</i>
<i>deleteq(q')</i>	error	if <i>isnewq(q)</i> then <i>newq</i> else <i>addq(deleteq(q), i)</i>

Capitolo 2

L'astrazione nella programmazione

È possibile costruire astrazioni su una qualunque classe sintattica, purchè le frasi di quella classe specifichino un qualche tipo di computazione.

Di seguito si esaminerà l'applicazione del principio di astrazione in sei classi sintattiche, in particolare:

1. Espressione → **astrazione di funzione**;
2. Comando → **astrazione di procedura**;
3. Controllo di sequenza → **astrazione di controllo**;
4. Accesso a un'area di memoria → **astrazione di selettore**;
5. Definizione di un dato → **astrazione di tipo**;
6. Dichiarazione → **astrazione generica**.

Tutte queste classi sintattiche sottointendono una computazione, infatti:

- L'astrazione di *funzione* include un'espressione da valutare.
- L'astrazione di una *procedura* include un comando da seguire.
- L'astrazione di *controllo* include delle espressioni di controllo dell'ordine di esecuzione delle istruzioni.
- L'astrazione di *selettore* include il calcolo dell'accesso ad una variabile.

- L'astrazione di *tipo* include un gruppo di operatori che definiscono implicitamente un insieme di valori.
- L'astrazione *generica* include una frase che sarà elaborata per produrre legami.

2.1 Astrazione di funzione

Un'astrazione di funzioni include un'espressione da valutare, e quando chiamata, darà un valore come risultato.

Un'astrazione di funzione è specificata mediante una definizione di funzione del tipo:

function $I(FP_1; \dots; FP_n)$ is E

dove I è un identificatore, $FP_1; \dots; FP_n$ sono parametri formali ed E è l'espressione da valutare.

In questo modo si lega I ad un'entità, l'astrazione di funzione, che ha la proprietà di dare un risultato ogni qualvolta che è chiamata con argomenti appropriati.

Una chiamata di funzione, $I(AP_1; \dots; AP_n)$ dove i parametri effettivi, $AP_1; \dots; AP_n$ determinano gli argomenti, ha due punti di vista:

1. Punto di vista dell'*utente*: la chiamata di una funzione trasforma gli argomenti in un risultato;
2. Punto di vista dell'*implementatore*: la chiamata valuta E , avendo precedentemente vincolato i parametri formali agli argomenti corrispondenti.
L'algoritmo codificato in E è di interesse per l'implementatore.

2.1.1 Anomalie di progetto dei linguaggi

Se l'astrazione di funzione include un'espressione E da valutare, è naturale attendersi che il corpo della funzione non includa dei comandi, come assegnazioni, istruzioni di salto, iterazioni, e altro, il cui effetto è quello di cambiare lo stato di un sistema, non di produrre valori.

Non sempre ciò accade, come si può osservare in questo esempio di funzione scritto in Pascal.

```

1  function power(x: real; n: integer) : real
2  begin
3      if n = 1 then power := x

```

```

4     else power := x * power(x, n - 1)
5 end
6

```

Nel corpo di questa funzione compaiono i comandi di assegnazione. Non è proprio possibile farne a meno, perchè per poter restituire un valore, in Pascal è necessario assegnare un valore ad una pseudo variabile, che ha lo stesso nome della funzione.

Il corpo della funzione è sintatticamente un comando, ma semanticamente è un'espressione, questa funzione, infatti, può essere invocata solo alla destra di operazioni di assegnazione.

Non tutti i linguaggi di programmazione includono comandi nel corpo di funzioni, infatti, nel linguaggio funzionale ML la funzione vista prima diventa:

```

1 function power(x: real; n: int) is
2   if n = 1 then x
3   else x * power(x, n-1)
4

```

Nel corpo della funzione definita in ML compare un'espressione condizionale la cui valutazione non modifica lo stato del sistema. Il motivo per il quale altri linguaggi di programmazione, come Pascal, Ada, o altro, permettono di avere comandi nel corpo di funzione è quello di poter sfruttare la potenza espressiva dell'assegnazione e dell'*iterazione* nella computazione di risultati. Diversamente saremmo costretti a esprimere ricorsivamente le espressioni da valutare. E la ricorsione, com'è noto, può essere fonte di inefficienze nell'uso delle risorse di calcolo.

Quindi, per ragioni di efficienza, molti linguaggi permettono di avere comandi nel corpo di funzioni e si lascia al programmatore il compito di utilizzare correttamente i comandi in modo da evitare che la funzione possa avere effetti collaterali, *side effect*, oltre quello di calcolare un valore.

Le funzioni in quanto astrazioni di espressioni, possono comparire ovunque occorra un'espressione. Pertanto, esse compaiono alla destra di operazioni di assegnazioni, ma anche al posto di parametri effettivi nelle chiamate di altre funzioni, laddove il valore potrebbe essere calcolato mediante un'espressione.

$$y := f(y, \text{power}(x, 2))$$

In molti linguaggi di programmazione è possibile definire dei parametri formali che sono riferimento ad una funzione.

```

1 function sommatoria(function F(R: real; M: integer): real; X: real; N:
integer) : real;

```

```

2   var I: integer;
3   var Sum: real;
4   begin
5       Sum := 0;
6       for I := 1 to N do Sum := Sum + F(x, I);
7       Sommatoria := Sum
8   end
9

```

Se la funzione F è la funzione *power* definita in precedenza, allora:

$$Y := \text{Sommatoria}(\text{power}, x, n)$$

calcola la sommatoria $\sum_{i=1}^n x^i$.

In alcuni linguaggi si separa il concetto di astrazione di funzione da quello di legame a un identificatore. Questo permette di passare come parametri delle funzioni anonime. Infine, le funzioni possono essere il risultato di valutazioni di espressioni o possono essere assegnate a variabili:

```

1   val cube = fn(x: real) => x * x * x

```

Riepilogando, in alcuni linguaggi di programmazione, le funzioni sono di *terza classe*, cioè possono essere solo chiamate, in altri sono di *seconda classe*, cioè possono essere passate come argomenti, mentre in altri linguaggi di programmazione esse sono di *prima classe* in quanto possono essere anche restituite come risultato della chiamata di altre funzioni o possono essere assegnate come valore a una variabile.

Cittadini di prima classe In generale, in programmazione una qualunque entità si dice *cittadino di prima classe* quando non è soggetta a restrizioni nel suo utilizzo. A tal proposito, osserviamo che i valori possono essere:

- *Denotabili*, se possono essere associati ad un nome;
- *Esprimibili*, se possono essere il risultato di un'espressione complessa;
- *Memorizzabili*, se possono essere memorizzati in una variabile.

2.2 Astrazione di procedura

Un'astrazione di procedura include un comando da eseguire, e quando chiamata, aggiornerà le variabili che rappresentano lo stato del sistema. Un'astrazione di

procedura è specificata mediante una definizione di procedura, del tipo:

$$\text{procedure } I(\text{FP}_1; \dots; \text{FP}_n) \text{ is } C$$

dove I è un identificatore, $\text{FP}_1; \dots; \text{FP}_n$ sono parametri formali, e C è il blocco di comandi da eseguire.

In questo modo si lega I all'astrazione di procedura, che gode della proprietà di cambiare lo stato del sistema quando chiamata con argomenti appropriati.

Data una chiamata di procedura, $I(\text{AP}_1; \dots; \text{AP}_n)$ dove $\text{AP}_1; \dots; \text{AP}_n$ sono i parametri effettivi, il punto di vista dell'*utente* è che la chiamata aggiornerà lo stato del sistema in modo dipendente dai parametri, mentre il punto di vista dell'*implementatore* è che la chiamata consentirà l'esecuzione del corpo di procedura C , avendo precedentemente vincolato i parametri formali agli argomenti corrispondenti. L'algoritmo in C è di interesse solo per l'implementatore.

In Pascal e in C le procedure sono *cittadini di seconda classe*.

L'astrazione di funzione o di procedura sono due tecniche di programmazione di supporto all'astrazione funzionale, intesa come tecnica di progettazione del software seconda la quale occorre distinguere la specifica di un'operatore dalla sua realizzazione.

Quale sia la tecnica di programmazione, se di funzione o di procedura, più opportuna da adottare dipende da:

- **Tipo di operatore progettato:**

- ha effetti collaterali, come accade nell'astrazione di procedura;
- non ha effetti collaterali, come accade nell'astrazione di funzione.

- **Limiti imposti dal linguaggio di programmazione.**

Ad esempio, l'operatore di *bubblesort* non modifica l'array passato in ingresso, ma ne restituisce uno ordinato, pertanto sarebbe più opportuno implementarlo ricorrendo ad un'astrazione di funzione. Tuttavia, non tutti i linguaggi di programmazione consentono di restituire come un tipo di dato complesso e quindi l'operatore *bubblesort* dovrà essere realizzato mediante un'astrazione di procedura.

2.3 Astrazione di controllo

L'astrazione di controllo si applica alla classe sintattica struttura di controllo. Le strutture di controllo definiscono l'ordine in cui le singole istruzioni o i gruppi

di istruzioni devono essere eseguiti.

Il linguaggio macchina generalmente fornisce due semplici meccanismi per governare il flusso di controllo delle istruzioni singole: l'**elaborazione in sequenza** e il **salto**.

Nei linguaggi assemblativi le istruzioni da eseguire in sequenza sono scritte l'una dopo l'altra: il salto è rappresentato da un'istruzione di jump:

jump to <indirizzo simbolico o label>

L'indirizzo simbolico è comunque un dettaglio di scarsa importanza per il programmatore: quello che conta è poter indicare le prossime istruzioni da eseguire. Per questa ragione i linguaggi di alto livello hanno introdotto strutture di controllo astratte come la **selezione**:

```
1  if cond then S1
2  else S2
```

corrisponde a:

```
1  jump on <cond> to A
2  S2
3  jump to B
4  A: S1
5  B: ...
```

Similmente sono state introdotte diverse strutture di controllo astratte per l'**iterazione**.

Inoltre per conservare gli indirizzi di ritorno dalle chiamate di funzione/procedura si è tradotta nella possibilità di effettuare chiamate **ricorsive**.

Di particolare interesse è l'attuale tendenza a offrire strutture di controllo iterative per quei dati astratti che sono collezioni omogenee, come insiemi, liste, array, ecc., di valori.

La struttura di controllo iterativa ha due parametri, il dato astratto e la variabile alla quale assegnare un valore contenuto nel dato astratto. Il meccanismo di visita della collezione di valori è incapsulato nel dato astratto, che dispone di operazioni lecite per consentire l'iterazione sulla collezione di oggetti.

Esempio : For-each in java

In Java è disponibile l'astrazione di controllo *for-each*, che permette di iterare su una collezione.

```
1  LinkedList<Integer> list = new LinkedList<Integer>();
2  ... // Inserimento elementi
3  for (Integer n : list) {
4      System.out.println(n);
```

```
5 }
```

L'operazione messa a disposizione di *LinkedList* per l'astrazione di controllo `foreach` è **iterator()**.

Se guardiamo alle strutture di controllo come espressioni che, quando valutate, definiscono l'ordine in cui eseguire dei comandi, possiamo specificare un'astrazione di controllo come segue:

$$\text{control } I(FP_1; \dots; FP_n) \text{ is } S$$

dove I è un identificatore di una nuova struttura di controllo, $FP_1; \dots; FP_n$ sono parametri formali, e S è un'espressione di controllo che definisce un ordine di esecuzione.

Esempio : Swap

```
1 control swap(boolean: cond, statement: S1, S2) is
2   if cond then
3     begin S1; S2 end
4   else
5     S2; S1
6   endif
```

Esempio : Alt

```
1 control alt_execution(statement: S1, S2, S3) is
2   if abnormal(S1) then
3     S2
4   else
5     S3
6   endif
```

abnormal è un predicato che ha in input un comando $S1$ e restituisce *true* se l'esecuzione di $S1$ è terminata in modo anomalo, *false* altrimenti.

I linguaggi di programmazione moderni mettono a disposizione dei meccanismi sofisticati per gestire le situazioni eccezionali come:

- errori aritmetici o di I/O;
- fallimento di precondizioni;
- condizioni imprecidibili.

Quando viene sollevata un'eccezione, essa dev'essere catturata e gestita.

Tipici approcci per localizzare il gestore dell'eccezione sono:

- cerca un gestore nel blocco (o sottoprogramma) corrente;
- se non c'è, forza l'uscita dell'unità corrente e solleva l'eccezione nell'unità chiamante;
- continua a risalire la gerarchia delle unità chiamanti finchè non trovi un gestore oppure non raggiungi il livello più alto, il main.

Quando viene trovato il gestore e l'eccezione è trattata:

- si riparte dall'unità contenente il gestore, *modello di terminazione*, come in Ada;
- si ritorna ad eseguire il comando che ha generato l'eccezione, *modello di ripresa*, come in PL/I.

I linguaggi di programmazione sequenziale utilizzano la sequenza, la selezione e la ripetizione per definire un **ordinamento totale** sull'esecuzione dei comandi di un programma.

I linguaggi di programmazione paralleli utilizzano ulteriori costrutti del flusso di controllo, come *fork*, *begin* o cicli *for* paralleli, per introdurre in modo esplicito un *ordinamento parziale* sull'esecuzione dei comandi.

Dato che il parallelismo è una forma di controllo della sequenza di esecuzione, l'astrazione di controllo è particolarmente importante nella programmazione parallela.

2.4 Astrazione di selettore

Un linguaggio di programmazione dispone di costrutti per poter accedere ad una variabile. Ad esempio in Pascal abbiamo:

```
1  var r: real;  
2  r := r * r;  
3  var vett: array[1..10] of char;  
4  vett[i] := vett[i+1];  
5
```

Lo stesso identificatore può essere usato sia come nome di un valore (legame dinamico fra identificatore e valore) e sia come un indirizzo (legame statico fra identificatore e locazione di memoria). Identicamente potremmo avere:

```

1  type rec = record
2      a: real,
3      b: real
4  end
5  var r: rec;
6  ...
7  r.b := r.a * 3;

```

Un accesso a una variabile restituisce un *riferimento* ad una variabile. Tuttavia se osserviamo il seguente codice Pascal:

```

1  type queue = ...;
2  var Aq: queue;
3  function first(q: queue): integer
4      // Restituisce il primo elemento della coda
5  i := first(Aq)

```

Ci accorgiamo che la chiamata $first(Aq)$ può comparire solo alla destra di un'assegnazione, perchè le funzioni restituiscono solo valori, mentre a sinistra $first(Aq) := 0$ dovrebbe restituire riferimenti ad aree di memoria.

In Pascal abbiamo dei selettori predefiniti dal progettista del linguaggio:

- F^* è un riferimento ad un puntatore di F ;
- $V[E]$ è un riferimento ad un elemento del vettore di V ;
- $R.A$ è un riferimento ad un elemento di un record.

ma il programmatore non ha modo di definire un nuovo selettore. In altri termini, non c'è la possibilità di definire astrazioni di selettore, che restituiscano l'accesso a un'area di memoria.

In C++ invece, possiamo definire astrazioni di selettore, infatti:

```

1  class Queue {
2      ...
3      public:
4          static int& first(Queue*);
5  };
6
7  Queue q*;
8  ...
9  int i = first(q);
10 first(q) *= first(q);
11 first(q)++;

```

Una possibile implementazione è la seguente:

```

1      class Queue {
2          int items[100];
3          int front, rear;
4          public:
5              ...
6          int& first(Queue* q) { return q->items[front] };
7      };

```

Flessibilità dei linguaggi L'espressività di un linguaggio di programmazione dipende direttamente da:

- I *meccanismi di composizione*, ossia la definizione di operazioni complesse a partire da quelle semplici.
- I *meccanismi di controllo di sequenza*, ossia la possibilità di stabilire ordini di esecuzione in modo semplice.
- I *valori* che il linguaggio permette di rappresentare e manipolare dati.

I linguaggi che supportano:

- L'astrazione funzionale sono flessibili nei meccanismi di composizione;
- L'astrazione di controllo sono flessibili nei meccanismi di controllo di sequenza;
- L'astrazione dati sono flessibili nella definizione e manipolazione di nuovi valori.

2.5 Astrazione dati

2.5.1 Tecniche di programmazione a supporto dell'astrazione dati

Le tecniche di programmazione a supporto dell'astrazione dati, che è una tecnica di progettazione del software, sono due:

- definizione di **tipi astratti**, cioè l'astrazione sulla classe sintattica tipo;
- definizione di *classi di oggetti*, cioè l'astrazione sulla dichiarazione di moduli dotati di stato locale.

In entrambi i casi occorre poter *incapsulare* la rappresentazione del dato con le operazioni lecite. Tuttavia,

- nel tipo di dato astratto, il modulo rende visibile all'utilizzatore sia un identificatore di tipo che degli operatori;
- nelle classi di oggetti, il modulo rende visibile all'utilizzatore solo gli operatori

Inoltre:

- nel tipo astratto T , i valori sono associati a variabili dichiarate di tipo T ;
- nella classe di oggetti C , i valori sono associati a oggetti ottenuti per istanziazione della classe di oggetti C .

Tipo concreto e tipo astratto I linguaggi ad alto livello mettono a disposizione del programmatore un nutrito gruppo di tipi predefiniti, detti **concreti**. Essi si distinguono in *primitivi* o *semplici* (i valori associati al tipo sono atomici) e *composti* o *strutturati* (i cui valori sono ottenuti per composizione di valori più semplici).

Tuttavia un linguaggio di programmazione sarà tanto più espressivo quanto più semplice sarà per il programmatore definire dei suoi tipi di dato a partire dai tipi di dato concreti disponibili. I tipi definiti dall'utente, detti *user defined types* UDT, sono anche detti **astratti**.

È bene ricordare che i tipi concreti sono messi a disposizione del linguaggio e tipi astratti sono definiti dall'utente. Ritornando all'astrazione di tipo, l'*espressione di tipo*, detto anche *tipo*, è il costrutto con cui alcuni linguaggi di programmazione consentono di definire un nuovo tipo. Ad esempio, in Pascal:

```
1  type Person = record
2      name: packed array[1..20] of char;
3      age: integer;
4      height: real
5  end
```

In questo caso si stabilisce esplicitamente una rappresentazione per i valori del tipo *Person* e implicitamente gli operatori applicabili a valori di quel tipo. Gli operatori del tipo *Person*, dovranno essere generici e non è possibile specificarli. Una **astrazione di tipo**, o *tipo astratto di dato* o semplicemente *dato astratto*, ha un corpo costituito da una espressione di tipo. Quando è valutata, l'astrazione di

tipo stabilisce sia una rappresentazione per un insieme di valori e sia le operazioni ad esse applicabili.

Analogamente a quanto detto per le altre astrazioni, l'astrazione di tipo potrà essere specificata come segue:

$$\text{type } I(FP_1; \dots; FP_n) \text{ is } T$$

dove I è un identificatore del nuovo tipo, $FP_1; \dots; FP_n$ sono parametri formali e T è un'espressione di tipo che specificherà la rappresentazione dei tipi di dato I e le operazioni ad esso applicabili. Analizziamo le astrazioni di tipo che alcuni linguaggi di programmazione (come C e Pascal) consentono di definire.

```
1  type complex = record
2      Re: real;
3      Im: real
4  end
```

consente di:

1. stabilire che *complex* è un identificatore di tipo;
2. associare una rappresentazione a *complex* espressa mediante tipi concreti già disponibili nel linguaggio.

Le operazioni associate al nuovo tipo *complex* sono tutte quelle che il linguaggio ha già previsto e per il tipo record, come assegnazione e selezione di campi.

I limiti di questa astrazione sono:

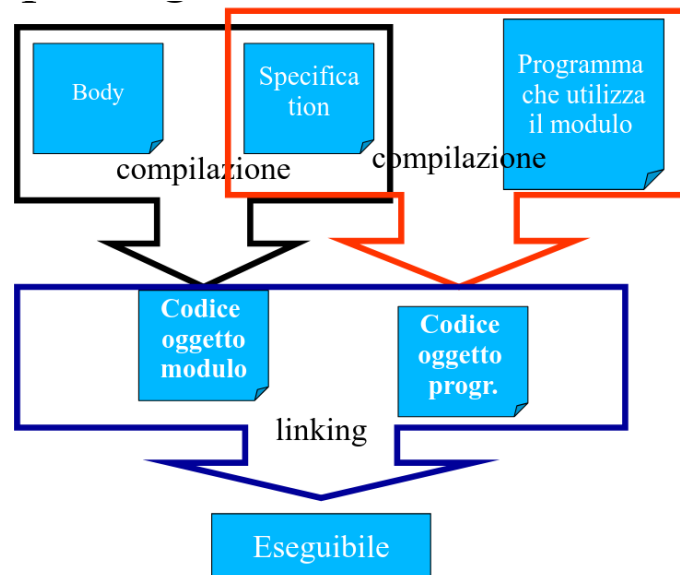
1. Il programmatore non può definire nuovi operatori specifici da associare al tipo.
2. Violazione del requisito di protezione: l'utilizzatore è consapevole della rappresentazione del tipo *complex*, sa che è un record e quali sono i campi, ed è in grado di operare mediante operatori non specifici del dato.
3. L'astrazione di tipo non è parametrizzata, la comunicazione con il contesto esterno non è ammessa.

Quindi, abbiamo detto che il programmatore non può definire nuovi operatori specifici da associare al tipo. Questo problema può essere risolto mediante un costrutto di programmazione che permette di **incapsulare**:

1. le rappresentazioni del dato;

2. gli operatori leciti.

Questo costrutto di programmazione è il **package**, ovvero *un gruppo di componenti dichiarate, come tipi, costanti, variabili, funzioni e persino sotto moduli*. Introduciamo quindi un costrutto package per supportare l'incapsulamento.



In Ada il modulo è chiamato package e si compone di due parti:

1. una **specification**, che interagisce con l'ambiente esterno: contiene le dichiarazioni di tipi, costanti, procedure e funzioni;
2. un **body**, che contiene, fra l'altro, l'implementazione di procedure e funzioni dichiarate nella specification, ed eventualmente una routine di inizializzazione del package.

A sua volta la specification si articola in due sottoparti:

1. **visible**: le entità dichiarate in questa parte possono essere rese note ad altre unità di programma per mezzo della clausola *use*;
2. **private**: le entità dichiarate in questa parte non possono essere né esportate e né dichiarate nel corpo.

La parte di specifica inizia con la parola chiave **package** seguita dall'identificatore del package e da **is**; seguono poi le dichiarazioni delle entità visibili e private.

```

1      package Type_complex is
2          type Complex is
3              record
4                  RL, IM: Real;
5              end record;
6          I: constant Complex := (0.0, 1.0)
7          function "+"(x, y: Complex) return Complex
8          ...
9      end Type_complex;
```

Questo è un esempio di specifica di un package che realizza il tipo astratto Complex. In questo non c'è una parte privata nella specifica, sicché un programma che fa uso di questo package può operare su variabili complesse mediante espressioni come la seguente:

$$C.IM := C.IM + 1.0$$

invece di ricorrere alla forma più astratta:

$$C := C + 1;$$

Per evitare il problema possiamo nascondere la struttura del tipo Complex nella parte privata.

```

1      package Type_complex is
2          type Complex is private;
3          function "+"(x, y: Complex) return Complex;
4          ...
5      private
6          type Complex is record
7              RL, IM: Real;
8          end record;
9          I: constant Complex := (0.0, 1.0)
10     end Type_complex;
```

Avendo dichiarato il tipo Complex nella parte privata non è più possibile inizializzare la costante I nella parte pubblica, in quanto non è ancora nota la rappresentazione di Complex.

Anche il corpo del package incomincia con la parola package seguita però dalla parola body, dall'identificatore e da is. Nel corpo sono fornite le implementazioni delle procedure e funzioni dichiarate nella corrispondente specification.

```

1      package body Type_complex is
2          function "+"(x, y: in Complex) return Complex is
3              begin
4                  return (x.RL + y.RL, x.IM + y.IM);
5              end "+";
6          ...
7      end Type_complex;

```

Ovviamente la struttura del tipo Complex è visibile alla parte body del package, quindi si potrà accedere ai campi RL e IM dei record Complex. Il package potrà essere utilizzato come segue:

```

1      with Type_complex; use Type_complex;
2      procedure main is
3          cpx1, cpx2: Complex;
4          begin
5              ...
6              cpx1 := cpx2 + I;
7              ...
8          end main;

```

Si osservi che sia nella specifica e sia nel corpo di Type_complex non c'è alcuna dichiarazione di variabili esterne a procedure e funzioni. Ciò vuol dire che questo package non è dotato di uno stato locale, cioè non definisce un oggetto. Per questa ragione il corpo del package non necessita di un "main": non si deve inizializzare un oggetto. Il package può in ogni caso avere un proprio main. Esso verrà specificato dopo le varie procedure e funzioni e sarà compreso fra un begin e l'end del package.

```

1      with Simple_io; use Simple_io;
2      package body Type_complex is
3          function "+"(x, y: in Complex) return Complex is
4              begin
5                  return(x.RL + y.RL, x.IM + y.IM);
6              end "+";
7          ...
8          begin
9              put("Main of package Type_complex");
10         end Type_complex;

```

L'esecuzione del main del package avverrà al momento in cui si importa il package mediante la clausola use. Si vedrà quindi visualizzare la frase "Main of package Type_complex".

Esempio : Astrazione di tipo per realizzare il dato astratto Pila

La specification è la seguente:

```
1 package Type_stack is
2   type Stack is private
3   procedure push(s: in out Stack; x: in integer);
4   procedure pop(s: in out Stack);
5   procedure top(s: in Stack; x: out Integer);
6   function empty(s: in Stack) return Boolean;
7 private
8   max: constant := 100;
9   type Stack is limited record
10    st: array(1..max) of Integer;
11    top: Integer range 0..max := 0;
12  end record;
13 end Type_stack;
```

Il body è il seguente:

```
1 procedure body Type_stack is
2   procedure push(s: in out Stack; x: in Integer) is
3     begin
4       s.top := s.top + 1;
5       s.st(s.top) := x;
6     end push;
7   procedure pop(s: in out Stack) is
8     begin
9       s.top := s.top - 1;
10    end pop;
11   procedure top(s: in Stack; x: out Integer) is
12     begin
13       x := s.st(s.top);
14     end top;
15   function empty(s: in Stack) return Boolean is
16     begin
17       return (s.top = 0);
18     end empty;
19 end Type_stack;
```

Le specifiche del tipo astratto Pila prevedono anche un costruttore CreaPila, che non ha un corrispondente in questa definizione del tipo astratto Stack. La ragione è che si stanno utilizzando costruttori impliciti forniti nel linguaggio Ada, come la dichiarazione di una variabile di tipo Stack. Tuttavia se un costruttore dovesse essere parametrizzato sarà necessario prevedere un metodo.

In Ada, definendo un tipo come *private* è possibile applicare su istanze di quel tipo tutti i metodi definiti nella parte pubblica della specifica, ma anche effettuare assegnazioni e confronti di (dis-)eguaglianza. Queste operazioni che il compilatore offre gratuitamente per un tipo privato devono necessariamente essere definite in modo generale, indipendentemente da come il tipo è poi definito.

Quindi saranno implementate semplicemente copiando o confrontando byte a byte le aree di memoria riservate a due dati dello stesso tipo dichiarato come privato. Questo, però, comporta dei problemi.

1. Le operazioni di assegnazione, confronto per eguaglianza e confronto per disequaglianza, potrebbero non far parte della specifica di un dato astratto. Quindi potrebbe non essere corretto offrirle all'utilizzatore del tipo dichiarato come privato.
2. La semantica delle operazioni potrebbe essere diversa da quella stabilita dal compilatore.

Ad esempio, consideriamo il seguente codice:

```
1      with Type_stack; use Type_stack;
2      procedure main is
3          st1, st2: Stack;
4          cmp: Boolean;
5      begin
6          push(st1, 1);
7          push(st2, 1);
8          push(st1, 2);
9          pop(st1);
10         cmp := st1 = st2;
11     end main;
```

Il valore di `cmp` è `False`. Infatti i corrispondenti top dei record `st1` e `st2` sarebbero identici, mentre non avrebbero gli stessi valori i corrispondenti campi `st`. Eppure i due stack sarebbero identici secondo la specifica algebrica di *equal*(*l*, *m*) data nel progetto.

Per evitare tutto ciò, è sufficiente dichiarare il tipo come **limited private**, che inibisce l'uso delle operazioni di assegnazione e confronto offerte per default dal compilatore.

```
1      package Type_stack is
2          type Stack is limited private
3          procedure push(s: in out Stack; x: in Integer);
4          procedure pop(s: in out Stack);
```

```

5      procedure top(s: in Stack; x: out Integer);
6      function empty(s: in Stack) return Boolean;
7      ...

```

In Ada è anche possibile definire **oggetti**, cioè moduli dotati di stato locale.

Esempio : Oggetto stack

La specifica è la seguente.

```

1      package Stack is
2          procedure push(x: in Integer);
3          procedure pop;
4          procedure top(x: out Integer);
5          function empty return Boolean;
6      end Stack;

```

Il body è il seguente.

```

1      package body Stack is
2          max: constant := 100;
3          type Table is array(1..max) of Integer;
4          st: Table;
5          top: Integer range 0..max := 0;
6
7          procedure push(x: in Integer) is
8              begin
9                  top := top + 1;
10                 st(top) := x;
11             end push;
12
13             procedure pop is
14                 begin
15                     top := top + 1;
16                 end pop;
17
18             procedure top(x: out Integer) is
19                 begin
20                     x := st(top);
21                 end top;
22
23             function empty return Boolean is
24                 begin
25                     return (top = 0);
26                 end empty;
27             end Stack;

```

Si potrà quindi utilizzare l'oggetto Stack nel modo seguente:

```

1      with Stack; use Stack;
2      procedure main is
3      begin
4          ...
5          push(1);
6          push(2);
7          pop
8          if empty then push(1);
9          ...
10     end main;

```

In generale, un oggetto è un insieme di variabili interne ad un modulo e manipolabili esternamente solo mediante gli operatori definiti nel modulo stesso. Da quanto visto finora, per poter definire più oggetti “simili”, cioè con medesima rappresentazione e stesso insieme di operatori, si è costretti a definire tanti moduli quanti sono gli oggetti che si vogliono usare nel programma. Tutti questi moduli differiranno solo per l’identificatore del modulo, cioè l’**identificatore dell’oggetto**.

Per evitare l’inconveniente di dover duplicare un modulo si può pensare di definire un **package generico** che identifica una *classe* di oggetti simili. I singoli oggetti sono poi ottenuti con il meccanismo di *istanziamento* della classe.

In Ada un package che specifica e implementa un singolo oggetto può essere facilmente trasformato in un *generic package*, che definisce una classe di oggetti, permettendo la parola *generic* alla dichiarazione del modulo.

```

1      generic
2      package Stack
3      procedure push(x: in Integer);

```

In questo modulo si definisce solo una *matrice* degli oggetti da creare. Per ottenere i singoli oggetti dobbiamo *istanziare* il generic package:

```

1      package st1 is new Stack
2      package st2 is new Stack

```

Queste due dichiarazioni sono processate in *fase di precompilazione*. In particolare, per ogni occorrenza di istanziamento, si sostituisce la stringa dell’istanziamento con il comando di importazione di un package avente come nome quello dell’istanza, ad esempio `with ST1; use ST1`.

Si genera un package non generico utilizzando il generic package come matrice. Esso si ottiene rimuovendo la parola *generic* e sostituendo il nome dell’istanza al posto del nome del package. Il package è poi compilato separatamente.

Nell’esempio specifico, la precompilazione genera due package distinti, che va-

riano solo nel nome del package, cioè nell'*identificatore dell'oggetto*.

Nota : Osservazione sul generic package

Di seguito alcune osservazioni sul generic package.

1. Il package generico **non è compilabile separatamente** in quanto il nome del package non identifica un oggetto.
2. La generazione di package distinti, uno per ogni istanziazione, comporta la creazione di molteplici copie dello stesso codice e questo porta ad una **inefficienza di spazio**.
3. La creazione dei legami al compile-time garantisce **efficienza in tempo**, poichè non è necessario effettuare computazioni di legami al run-time.
4. L'ambiguità dovuta alle molteplici occorrenze di metodi con stesso identificatore richiede il ricorso, nel programma utilizzatore, alla notazione <identificatore di oggetto>.<nome metodo>, ad esempio ST1.push(10)

2.6 Astrazione nella dichiarazione di modulo

La precedente definizione di una classe corrisponde a una particolare forma di astrazione, quella della classe sintattica '*dichiarazione di un modulo*'.

L'operazione di istanziazione corrisponde alla invocazione di questa astrazione ed ha l'effetto di creare legami. In particolare, si crea un legame fra

- identificatore dell'oggetto;
- nome della classe.

Pertanto la definizione di una classe corrisponde a una particolare forma di *astrazione generica*, cioè di astrazione applicata alla classe sintattica dichiarazione.

Si è già visto che un modulo può essere usato per incapsulare un *tipo astratto* con gli operatori leciti. Vediamo se ha senso definire *generic* un modulo di questa fatta e poi istanziarlo.

Riepilogando, se in fase di progettazione si identifica l'esigenza di un dato astratto, in fase realizzativa si può:

1. *Definire un oggetto*: la scelta è appropriata nel caso in cui si necessita di una sola occorrenza del dato astratto;

2. *Definire un tipo astratto*: l'astrazione riguarda la classe sintattica tipo;
3. *Definire una classe*: l'astrazione riguarda la dichiarazione di un modulo (dotata di stato locale).

In tutti i casi la rappresentazione del dato astratto viene nascosta e la manipolazione dei valori è resa possibile solo mediante operazioni fornite allo scopo. Tuttavia, ci sono delle differenze fra tipo astratto e classe di oggetti.

Capitolo 3

Paradigma Object Oriented

3.1 Gli oggetti

Si deve osservare che nella programmazione imperativa è possibile definire degli oggetti, tuttavia

1. il loro utilizzo è legato all'auto disciplina dei programmatori e non viene in alcun modo forzato;
2. gli oggetti **non sono cittadini di prima classe**.

Confrontando il paradigma orientato a oggetti rispetto a quello imperativo si può dire che esso costituisce:

1. Un'*evoluzione* in quanto permette agli oggetti di essere cittadini di prima classe;
2. Una *rivoluzione* in quanto gli oggetti assumono un ruolo **fondamentale** nella progettazione e nella programmazione. L'*information hiding* e l'*incapsulamento* sono principi cardine nel paradigma orientato agli oggetti.

Gli oggetti incapsulano uno *stato* e un *comportamento*. Lo stato è identificato dal contenuto di una certa area di memoria. Il comportamento è definito da una collezione di procedure e funzioni, detti *metodi*, che possono operare sulla rappresentazione dell'area di memoria associata all'oggetto.

Da una prospettiva di progetto, gli oggetti modellano le *entità* presenti nel dominio dell'applicazione.

Esempio

In un gioco elettronico che usa palla, si può pensare alla palla come un oggetto dotato di uno:

- **Stato:** dimensione, posizione in uno spazio di riferimento, ecc.
- **Comportamento:** una palla può “apparire” o “scompare” dallo schermo, può muoversi, rimbalzare, ecc.

Le variabili e i metodi definiti nell’oggetto “palla” stabiliscono lo stato e il comportamento che sono rilevanti all’uso della palla nel gioco elettronico.

3.1.1 Identificatore di oggetto

Un oggetto ha la sua *identità*, cioè è riconoscibile indipendentemente dal suo stato corrente. Per questo ogni oggetto ha un *identificatore di oggetto*, detto object identifier OID, che lo identifica univocamente. In alcuni contesti gli OID sono anche detti *riferimenti*.

Un identificatore di oggetto è *immutabile*, cioè non può essere modificato da una qualche operazione di programmazione. Cambiare l’OID di un oggetto equivale alla cancellazione dell’oggetto e alla creazione di un altro oggetto con lo stesso dato.

Normalmente gli OID sono assegnati in modo automatico agli oggetti, sicché non hanno un significato nel mondo reale. In molti ambienti di programmazione object-oriented, l’OID corrisponde all’indirizzo dell’area di memoria che conserva lo stato dell’oggetto.

3.2 Le classi

Una *classe* è la descrizione di una famiglia di oggetti che condividono la stessa struttura (gli attributi) e il medesimo comportamento (operazioni). Nella programmazione OO ogni oggetto è un’istanza di una classe, cioè un oggetto non può essere definito se non si definisce la classe di appartenenza. Analogamente nella modellazione OO le istanze esistono in quanto ci sono le loro astrazioni.

Idealmente una classe è una realizzazione di un dato astratto. Questo significa che i dettagli della realizzazione sono normalmente nascosti.